

Git и терминал. Шпаргалка

Глоссарий

- **VCS (Version Control System / Система контроля версий)** - Система для отслеживания изменений в файлах и координации работы нескольких людей.
- **Git** - Распределенная система контроля версий, созданная Линусом Торвальдсом.
- **Терминал (Terminal / Консоль)** - Текстовый интерфейс для взаимодействия с операционной системой.
- **Git Bash** - Эмулятор терминала для Windows, предоставляющий командную строку в стиле Linux и Git.
- **CLI (Command Line Interface / Интерфейс командной строки)** - Способ взаимодействия с программой путем ввода текстовых команд.
- **GUI (Graphical User Interface / Графический интерфейс пользователя)** - Визуальный способ взаимодействия с программой.

Основные понятия Git

- **Репозиторий (Repository / Репозитив, Хранилище)** - Каталог (папка), в котором Git отслеживает все изменения файлов. Содержит служебную папку .git.
- **Индекс (Index / Область подготовленных файлов, Staging Area)** - Промежуточная область между рабочим каталогом и историей коммитов. Содержит изменения, которые будут включены в следующий коммит.
- **Коммит (Commit / Фиксация)** - Снимок состояния файлов в определенный момент времени, сохраненный в истории репозитория. Имеет уникальный хеш (например, a1b2c3d).
- **Родительский коммит (Parent Commit)** - Коммит, на основе которого был создан текущий. Формирует цепочку истории.
- **Ветка (Branch)** - Указатель на определенный коммит. Позволяет вести параллельные линии разработки. Основная ветка по умолчанию — master или main.
- **HEAD** - Специальный указатель, который показывает на какой вы сейчас ветке и коммите (текущее состояние рабочего каталога).
- **Working Directory / Working Tree (Рабочий каталог)** - Текущая папка на вашем компьютере, где вы работаете с файлами репозитория.

Состояния файлов в Git

1. **Untracked (Неотслеживаемый)** — файл существует в рабочем каталоге, но Git не отслеживает его изменения.
2. **Staged (Подготовленный)** — файл добавлен в индекс и готов к коммиту.
3. **Modified (Измененный)** — файл, отслеживаемый Git, был изменен в рабочем каталоге, но изменения еще не добавлены в индекс.
4. **Unmodified / Committed (Зафиксированный)** — файл не изменялся с момента последнего коммита.

Операции и процессы

- **Clone (Клонирование)** - Создание локальной копии удаленного репозитория на ваш компьютер.
- **Fork (Форк, Ответвление)** - Копия репозитория на GitHub/GitLab на ваш аккаунт, позволяющая вносить изменения независимо от оригинального проекта.
- **Pull Request / Merge Request (PR/MR / Запрос на слияние)** - Механизм на платформах (GitHub, GitLab) для предложения своих изменений из форка или ветки в основной репозиторий.
- **Merge (Слияние)** - Процесс объединения изменений из одной ветки в другую. Создает специальный коммит слияния.
- **Rebase (Перебазирование)** - Альтернатива слиянию: перемещает или комбинирует последовательность коммитов на новый базовый коммит, создавая более линейную историю.
- **Pull (Получение изменений)** - Команда для загрузки изменений из удаленного репозитория и их слияния с текущей веткой (`git pull = git fetch + git merge`).
- **Push (Отправка изменений)** - Команда для выгрузки ваших локальных коммитов в удаленный репозиторий.
- **Fetch (Загрузка изменений)** - Команда для загрузки последних изменений из удаленного репозитория **без** их слияния с вашими файлами.
- **Checkout (Переключение)** - Команда для переключения между ветками или восстановления файлов из коммита (в старом Git). В новом Git для создания ветки используют `git switch`.
- **Stash (Временное сохранение)** - Временное сохранение незакоммиченных изменений в «карман», чтобы очистить рабочую директорию.

- **Конфликт слияния (Merge Conflict)** - Ситуация, когда Git не может автоматически объединить изменения из разных веток, так как они затрагивают одни и те же строки в файле. Требуется ручное разрешение.
- **Отслеживание (Tracking)** - Связь между локальной веткой и удаленной веткой, позволяющая использовать короткие команды `git pull` и `git push`.
- **Origin** - Стандартное имя (псевдоним) для удаленного репозитория, с которого был клонирован проект.
- **Upstream (Восходящий репозиторий)** - Оригинальный репозиторий, от которого был сделан форк (в контексте работы с форками).
- **Tag (Тег)** - Указатель на конкретный коммит, обычно используется для разметки версий (v1.0, v2.1).
- **.gitignore** - Специальный файл, в котором перечислены шаблоны файлов и папок, которые Git должен игнорировать и не отслеживать.

Базовые команды терминала (GitBash / Linux)

Навигация по файловой системе

- **pwd** (print working directory) – Показать текущую директорию (полный путь).
- **ls** (list) – Показать содержимое текущей директории.
 - **ls -l** – Подробный список (права, размер, дата).
 - **ls -a** – Показать ВСЕ файлы, включая скрытые (начинающиеся с .).
 - **ls -la** – Комбинация: подробный список всех файлов.
- **cd** <путь> (change directory) – Перейти в указанную директорию.
 - **cd ~** или просто **cd** – Перейти в домашнюю директорию.
 - **cd ..** – Подняться на уровень выше (в родительскую папку).
 - **cd -** – Вернуться в предыдущую директорию.
- **clear** или **Ctrl + L** – Очистить экран терминала.

Работа с файлами и директориями

- **touch** <имя_файла> – Создать пустой файл или обновить время его изменения.
- **mkdir** <имя_папки> (make directory) – Создать новую папку.
 - **mkdir -p path/to/new/folder** – Создать целую иерархию папок (с родительскими).
- **cp** <откуда> <куда> (copy) – Копировать файл или папку.
 - **cp file.txt dir/** – Скопировать файл в директорию dir.
 - **cp -r dir1/ dir2/** – Копировать директорию dir1 рекурсивно (со всем содержимым) в dir2.
- **mv** <откуда> <куда> (move) – Переместить или **переименовать** файл/папку.
 - **mv old.txt new.txt** – Переименовать файл.
 - **mv file.txt ~/Documents/** – Переместить файл.
- **rm** <имя> (remove) – Удалить файл (необратимо!).
 - **rm -r <имя_папки>** – Удалить папку и всё её содержимое рекурсивно.
 - **rm -f <имя>**** – Удалить принудительно (force), без подтверждения.
- **cat** <имя_файла> (concatenate) – Показать содержимое файла.
- **less** <имя_файла> – Просмотреть содержимое файла постранично (прокрутка: стрелки, выход: q).
- **head** <имя_файла> – Показать первые 10 строк файла.
 - **head -n 20 file.txt** – Показать первые 20 строк.

- **tail** <имя_файла> – Показать последние 10 строк файла.
 - **tail -f file.log** – Показывать конец файла в реальном времени (следить за логами).

Поиск и информация

- **find** <путь> -name "<шаблон>" – Найти файлы по имени.
 - **find . -name "*.js"** – Найти все .js файлы в текущей директории и поддиректориях.
- **grep** "<текст>" <имя_файла> – Найти строки, содержащие текст, в файле.
 - **grep -r "TODO" .** – Рекурсивно искать слово "TODO" во всех файлах текущей директории.
 - **grep -n "error" app.log** – Искать "error" с выводом номеров строк.
- **which** <команда> – Показать полный путь к исполняемому файлу команды (например, which git).
- **history** – Показать историю введенных команд.
 - **!**<номер>**** – Выполнить команду из истории под номером (например, !155).
 - **Ctrl + R** – **Поиск по истории** (нажать и начать вводить часть команды).

Полезные сочетания клавиш

- **Tab** – Автодополнение пути или имени команды.
- **Ctrl + C** – Прервать выполнение текущей команды.
- **Ctrl + D** – Закрывает терминал (или отправить EOF).
- **Ctrl + Z** – Приостановить процесс (вернуть: fg или bg).
- **Ctrl + A / Ctrl + E** – Переместить курсор в **начало** / **конец** строки.
- **Ctrl + U / Ctrl + K** – Удалить текст от курсора до **начала** / **конца** строки.
- **Ctrl + W** – Удалить слово перед курсором.
- **↑ / ↓** – Перемещаться по истории команд.

Важно для Git Bash на Windows:

- Пути разделяются **прямым слешем (/)**, как в Linux: /c/Users/YourName/Projects/
- Для доступа к дискам используйте /c/, /d/ и т.д.
- Многие команды Windows (например, dir, copy) **не работают**. Используйте их Unix-аналоги (ls, cp).

Основные команды Git

1. Начало работы (создание и клонирование)

- **git init** – Инициализировать новый локальный репозиторий в текущей папке.
- **git clone <url-репозитория>** – Создать локальную копию удаленного репозитория.
 - **git clone https://github.com/user/repo.git** – Клонировать по HTTPS.
 - **git clone git@github.com:user/repo.git** – Клонировать по SSH.

2. Текущее состояние и история

- **git status** – Показать состояние рабочего каталога и индекса (какие файлы изменены, добавлены, не отслеживаются).
- **git log** – Показать историю коммитов.
 - **git log --oneline** – Краткая история (хеш и сообщение в одну строку).
 - **git log --graph --oneline** – История с визуализацией ветвления.
 - **git log -p** – Показать историю с изменениями в коде (diff).
- **git diff** – Показать **незакоммиченные** изменения в рабочем каталоге (отличие от индекса).
 - **git diff --staged** – Показать изменения, которые уже добавлены в индекс (staged).
- **git show <хэш_коммита>** – Показать информацию и изменения конкретного коммита.

3. Работа с индексом (Staging Area)

- **git add <файл>** – Добавить конкретный файл в индекс.
- **git add .** или **git add -A** – Добавить **все** измененные и новые файлы в индекс.
- **git add -u** – Добавить в индекс только измененные и удаленные файлы (новые не добавит).
- **git rm <файл>** – Удалить файл из рабочего каталога **и из индекса Git**.
 - **git rm --cached <файл>** – Удалить файл из индекса Git, но **оставить в рабочей папке** (чтобы добавить в .gitignore).
- **git restore <файл>** – **Отменить изменения** в рабочем каталоге (вернуть к состоянию последнего коммита или индекса).
 - **git restore --staged <файл>** – Убрать файл из индекса (unstaged), но сохранить изменения в файле.

- **git mv** <старое_имя> <новое_имя> – Переименовать или переместить файл с учетом версионности.

4. Создание коммитов

- **git commit -m** "<сообщение>" – Зафиксировать изменения в индексе, создав новый коммит с сообщением.
- **git commit -a -m** "<сообщение>" – Добавить в индекс все измененные/удаленные отслеживаемые файлы **и сразу сделать коммит** (новые файлы не добавит).
- **git commit --amend** – **исправить** последний коммит (добавить изменения в индекс, изменить сообщение). **Перезаписывает историю!**

5. Работа с ветками

- **git branch** – Показать список локальных веток (текущая помечена *).
 - **git branch -a** – Показать все ветки (локальные и удаленные).
- **git branch** <имя_ветки> – Создать новую ветку от текущего коммита.
- **git checkout** <имя_ветки> – Переключиться на указанную ветку.
 - **git checkout -b** <имя_ветки> – **Создать и сразу переключиться** на новую ветку (классический способ).
- **git switch** <имя_ветки> – **Новая команда** для переключения на ветку (более понятная альтернатива checkout).
 - **git switch -c** <имя_ветки> – Создать и переключиться (аналог checkout -b).
- **git merge** <имя_ветки> – Влить указанную ветку в текущую.
- **git rebase** <ветка_или_коммит> – Переместить/перебазировать коммиты текущей ветки на указанную.
- **git branch -d** <имя_ветки> – Удалить локальную ветку (безопасно, если изменения влиты).
 - **git branch -D** <имя_ветки> – Принудительно удалить локальную ветку (даже если не влита).

6. Работа с удаленным репозиторием (Remote)

- **git remote -v** – Показать список удаленных репозиторий (с URL).
- **git remote add** <имя> <url> – Добавить удаленный репозиторий (по умолчанию origin).
 - **git remote add origin https://github.com/...**

- **git fetch <имя_удаленного_репо>** – Скачать все изменения из удаленного репозитория, **но не вливать** в локальные ветки.
 - **git fetch origin** – Обновить информацию об удаленных ветках origin.
- **git pull <remote> <ветка>** – Скачать изменения из удаленной ветки **и сразу влить** в текущую локальную ветку.
 - **git pull origin main** – Чаще всего используется в этом виде.
 - **git pull --rebase** – Стянуть изменения с перебазируванием (избегает лишних merge-коммитов).
- **git push <remote> <ветка>** – Отправить локальные коммиты в удаленную ветку.
 - **git push origin main** – Отправить ветку main в origin.
 - **git push -u origin <ветка>** – Отправить ветку и установить связь (upstream), чтобы в дальнейшем можно было просто писать git push.
- **git push --force** или **git push --force-with-lease** – **Принудительная** отправка (перезаписывает историю на удаленном репо). **Опасно! Использовать с крайней осторожностью.**

7. Временное сохранение изменений (Stash)

- **git stash push -m "<сообщение>"** – Временно отложить (спрятать) все незакоммиченные изменения.
- **git stash list** – Показать список всех "спрятанных" изменений.
- **git stash pop** – Вернуть последние отложенные изменения и удалить их из списка stash.
- **git stash apply** – Вернуть последние отложенные изменения, но **оставить** их в списке stash.
- **git stash drop** – Удалить последние отложенные изменения из списка stash.

8. Отмена действий и "спасение"

- **git restore <файл>** – Откатить изменения в файле до состояния индекса или последнего коммита.
- **git reset <коммит>** – **Переместить** указатель текущей ветки на другой коммит (и откатить индекс).
 - **git reset --soft <коммит>** – Откатить коммит, но оставить изменения в индексе.

- **git reset --mixed <коммит>** – Откатить коммит и индекс (по умолчанию). Изменения останутся в рабочем каталоге.
 - **git reset --hard <коммит>** – **Полный откат** (опасно): коммит, индекс и рабочий каталог приведутся к состоянию указанного коммита.
- **git revert <хэш_коммита>** – Создать **новый коммит**, который отменяет изменения указанного коммита. Безопасный способ отмены для публичной истории.

9. Полезные мелочи (просмотр, теги, игнорирование)

- **git blame <файл>** – Показать, кто и в каком коммите менял каждую строку файла.
- **git tag** – Показать список тегов.
 - **git tag -a v1.0 -m "Версия 1.0"** – Создать аннотированный тег на текущем коммите.
 - **git push origin --tags** – Отправить все теги на удаленный репозиторий.
- **.gitignore** – Файл, в котором указываются шаблоны игнорируемых файлов (логи, сборки, настройки IDE и т.д.).

Типовой рабочий процесс (цикл)

1. **Начало дня/задачи:** **git pull --rebase origin main** (стянуть свежие изменения)
2. **Создать ветку:** **git switch -c feature/my-new-feature**
3. **Работа с файлами:** редактирование, сохранение...
4. **Проверить статус:** **git status**
5. **Добавить изменения:** **git add .** или **git add <конкретные_файлы>**
6. **Закоммитить:** **git commit -m "Add amazing feature"**
7. **Повторить п.3-6** по необходимости.
8. **Отправить на сервер:** **git push -u origin feature/my-new-feature** (первый раз), затем просто **git push**
9. **Создать Pull Request** на GitHub/GitLab.
10. После мержа в main: удалить локальную ветку **git branch -d feature/my-new-feature** и обновить main.